

objectives

1. Define JSX

JSX (JavaScript XML) is a syntax extension for JavaScript that allows developers to write HTML-like code inside JavaScript. It makes React components easier to read and write. JSX is converted into regular JavaScript by tools like Babel before execution.

2. Explain ECMAScript (ES6)

ECMAScript (ES) is the standard specification on which JavaScript is based. **ES6 (ECMAScript 2015)** introduced modern features like `let`, `const`, arrow functions, classes, template literals, destructuring, promises, and modules. These features make JavaScript more powerful, readable, and easier to maintain.

3. Explain `React.createElement()`

`React.createElement()` is the function React uses to create React elements. It accepts the element type, properties, and children, then returns a React element object. JSX is automatically transformed into `React.createElement()` calls during compilation.

4. Explain how to create React nodes with JSX

React nodes can be created by writing HTML-like syntax directly in JSX. JSX is easier to understand than using `React.createElement()` manually and can include components, elements, and expressions.

5. Define how to render JSX to DOM

To display JSX on a web page, it must be rendered into the DOM using ReactDOM. In React 18, `createRoot()` is used to render the root component.

6. Explain how to use JavaScript expressions in JSX

JavaScript expressions can be embedded inside JSX using **curly braces `{}`**. This allows dynamic values, calculations, function calls, and variables to be displayed in the UI.

7. Explain how to use inline CSS in JSX

Inline CSS in JSX is written as a JavaScript object using the `style` attribute. CSS property names use **camelCase** instead of hyphens, and values are enclosed in quotes or written as numbers.

Code

```
4 function App() {
36
37   return (
38     <div className="container">
39       <h1> Office Space Rental App</h1>
40
41       <h2>Featured Office</h2>
42
43       <div className="featured-card">
44         <img src={office.image} alt={office.name} />
45
46         <div className="info">
47           <h3>{office.name}</h3>
48
49           <p>
50             <strong>Rent:</strong>{" "}
51             <span
52               className={office.rent < 60000 ? "lowRent" : "highRent"}
53             >
54               ₹{office.rent}
55             </span>
56           </p>
57
58           <p>
59             <strong>Address:</strong> {office.address}
60           </p>
61         </div>
62       </div>
63
64       <h2>Available Offices</h2>
65
66       <div className="office-container">
67         {officeList.map((item, index) => (
68           <div className="office-card" key={index}>
69             <img src={item.image} alt={item.name} />
```

Installing dependencies with npm...

```
60     </p>
61   </div>
62 </div>
63
64 <h2>Available Offices</h2>
65
66 <div className="office-container">
67   {officeList.map((item, index) => (
68     <div className="office-card" key={index}>
69       <img src={item.image} alt={item.name} />
70
71       <h3>{item.name}</h3>
72
73       <p>
74         <strong>Rent:</strong>{" "}
75         <span
76           className={item.rent < 60000 ? "lowRent" : "highRent"}
77         >
78           ₹{item.rent}
79         </span>
80       </p>
81
82       <p>{item.address}</p>
83     </div>
84   ))}
85 </div>
86 </div>
87
88 }
89
90 export default App;
```

Installing dependencies with npm...

Output

